

System Design & Networking Fundamentals

— Part 2

What Really Happens When You Type a Website URL?

We type `google.com` or `example.com` every day, but computers do not understand domain names the way humans do. Devices communicate using `IP addresses`, so the browser first needs to discover which IP address belongs to the requested domain.

That is where `DNS (Domain Name System)` comes in.

> Think of DNS as the phonebook of the internet: it translates a memorable domain name into an IP address that a computer can use.

! [DNS overview and lookup hierarchy]
(./image.png)

📖 Table of Contents

1. [The Complete Journey]
(#the-complete-journey)
2. [Step-by-Step DNS Lookup]
(#step-by-step-dns-lookup)
3. [The DNS Hierarchy] (#the-dns-hierarchy)
4. [Recursive and Iterative Queries]
(#recursive-and-iterative-queries)
5. [Common DNS Record Types]
(#common-dns-record-types)
6. [Why DNS Is Fast] (#why-dns-is-fast)
7. [Reliability and Security]
(#reliability-and-security)
8. [What Happens After DNS?]
(#what-happens-after-dns)
9. [Key Takeaways] (#key-takeaways)

The Complete Journey

At a high level, a DNS lookup follows this path:

""text

Domain Name



Browser and OS Caches



Recursive DNS Resolver



Root Name Server



TLD Name Server



Authoritative Name Server



IP Address



Web Server

""

In practice, caching often allows the browser or

resolver to return the answer before every DNS layer needs to be contacted.

Step-by-Step DNS Lookup

Suppose you enter `www.example.com` in your browser.

① The Browser Parses the URL

The browser identifies the important URL components:

“text

<https://www.example.com/products?id=42>

└──┴──┘

└──────────┴──────────┘

└──┴──┘

scheme

host

path and query

”

DNS is responsible for resolving the ****host****,
'www.example.com'.

(2) The Browser Checks Its DNS Cache

The browser checks whether it recently resolved this domain. If a valid cached result exists, it can reuse the IP address immediately.

(3) The Operating System Checks Its Cache

If the browser has no answer, it asks the operating system. The OS may check:

- Its local DNS cache
- A local hosts file
- The configured DNS resolver

(4) The Recursive Resolver Looks for the Answer

The request is sent to a ****recursive DNS**

resolver**, usually operated by an ISP, company, or public DNS provider.

Examples of public resolvers include:

- Cloudflare: `1.1.1.1`
- Google Public DNS: `8.8.8.8`
- Quad9: `9.9.9.9`

The resolver checks its own cache first. If it already has a valid record, it returns the answer without querying the wider DNS hierarchy.

(5) The Resolver Asks a Root Name Server

If the result is not cached, the resolver starts near the top of the DNS hierarchy.

The **root name server** does not normally know the final IP address. It directs the resolver to the appropriate Top-Level Domain server—for example, the name servers responsible for `.com`.

6 The Resolver Asks the TLD Name Server

The ****TLD (Top-Level Domain) name server**** manages information for a domain extension such as:

- `.com`
- `.org`
- `.net`
- `.in`

It tells the resolver which authoritative name servers are responsible for `example.com`.

7 The Resolver Asks the Authoritative Name Server

The ****authoritative name server**** stores the DNS records for the domain. It returns the requested record, such as:

```text

www.example.com → 93.184.216.34

```

The answer may also involve following one or more aliases, called 'CNAME' records, before reaching an IP address.

8 The IP Address Is Returned

The resolver returns the result to the operating system, which passes it to the browser. The result is cached according to its **TTL (Time to Live)** so future requests can be answered faster.

9 The Browser Connects to the Server

Now that it has an IP address, the browser can begin connecting to the destination server. DNS has completed its primary job; transport security and HTTP communication happen next.

The DNS Hierarchy

DNS is a distributed, hierarchical system. No single server needs to store every domain on the internet.

Root Name Servers

Root name servers sit at the top of the hierarchy. They point resolvers toward the correct TLD name servers.

TLD Name Servers

TLD servers know which authoritative name servers handle domains within their extension.

``text

.com → authoritative servers for example.com

.org → authoritative servers for example.org

``

Authoritative Name Servers

These servers contain the official DNS records for a domain and provide the final authoritative answer.

This distributed design improves:

- Scalability
- Availability
- Fault tolerance
- Administrative independence

Recursive and Iterative Queries

These two terms describe who is responsible for finding the final answer.

Recursive query

The client asks a recursive resolver for a complete answer:

```text

Browser → Resolver: "What is the IP address for www.example.com?"

```

The resolver performs the required lookups and returns the final result or an error.

Iterative Query

The resolver asks DNS servers one at a time. Each server returns either an answer or a referral to another server:

```text

Resolver → Root: "Where is www.example.com?"

Root → Resolver: "Ask the .com servers."

```

The browser typically makes a recursive request,

while the resolver performs iterative queries through the DNS hierarchy.

Common DNS Record Types

DNS can return more than an IPv4 address.

Record	Purpose	Example
---	---	---
`A`	Maps a name to an IPv4 address	`example.com → 93.184.216.34`
`AAAA`	Maps a name to an IPv6 address	`example.com → 2001:db8::1`
`CNAME`	Makes one hostname an alias of another	`www.example.com → example.com`
`MX`	Identifies mail servers for a domain	`example.com → mail.example.com`
`NS`	Identifies authoritative name servers	`example.com → ns1.example.net`
`TXT`	Stores text used for verification and policies	SPF or domain verification

| 'SOA' | Describes the DNS zone and its administrative data | Primary server and timing values |

Why DNS Is Fast

A complete lookup may contact several systems, but users usually experience it in milliseconds because DNS results are cached at multiple levels:

```text

Browser Cache

↓ miss

Operating System Cache

↓ miss

Recursive Resolver Cache

↓ miss

DNS Hierarchy

```

TTL (Time to Live)

Each DNS record has a TTL that tells caches how long they may reuse the answer.

- A **long TTL** reduces DNS traffic and improves cache hit rates.
- A **short TTL** allows changes to propagate sooner but causes more lookups.

Choosing a TTL is a trade-off between performance and how quickly DNS changes need to take effect.

Negative Caching

DNS can also cache unsuccessful results, such as a response stating that a domain does not exist. This prevents repeated queries for the same invalid name.

Reliability and Security

Redundancy

Domains normally use multiple authoritative name servers. DNS infrastructure also commonly uses ****anycast****, allowing the same IP address to be served from many geographic locations.

DNS Failure Scenarios

A lookup can fail because of:

- An expired or incorrect DNS record
- An unavailable resolver or authoritative server
- A DNS timeout
- A misconfigured delegation
- A domain that does not exist (NXDOMAIN)

DNS Security

Traditional DNS was not designed to

authenticate responses. Important improvements include:

- ****DNSSEC****: Adds cryptographic signatures so resolvers can verify DNS data.
- ****DNS over HTTPS (DoH)****: Encrypts DNS queries inside HTTPS.
- ****DNS over TLS (DoT)****: Encrypts DNS queries using TLS.

DNSSEC provides authenticity and integrity; DoH and DoT provide privacy between the client and its resolver. They solve different problems.

What Happens After DNS?

Once the browser receives an IP address, the journey continues:

``text

IP Address



Route packets across the network



Establish a TCP connection or use QUIC



Perform a TLS handshake for HTTPS



Send an HTTP request



Receive the HTTP response



Render the page

...

These steps introduce the next foundational concepts:

- IP addressing and routing
- TCP and UDP
- QUIC and HTTP/3
- TLS and HTTPS
- HTTP requests and responses
- Load balancers, CDNs, and web servers

Key Takeaways

- DNS translates human-friendly domain names into machine-friendly IP addresses.
 - Browsers, operating systems, and recursive resolvers all cache DNS results.
 - A cache miss may trigger queries to root, TLD, and authoritative name servers.
 - DNS is distributed and hierarchical, so no single server stores every answer.
 - TTL controls how long a DNS response can be cached.
 - After DNS resolution, the browser still needs to establish a connection, secure it, request the resource, and render the response.
- > ⚡ A journey across several layers of internet infrastructure usually happens in just a few milliseconds.

Continue the Learning Path

- **Previous:** Part 1 — Client-Server Architecture in [fundamentals.md] (/fundamentals.md)
- **Next:** What happens after DNS gives us the IP address?